

MagneticTB manual

Zeying Zhang

September 7, 2026

Contents

1	Features	2
2	Installation, updates, and uninstallation	2
2.1	Installing and loading the package	2
2.2	Opening the documentation	2
2.3	Updating	3
2.4	Uninstalling	3
3	Using MagneticTB	4
3.1	Obtaining symmetry operations	4
3.2	Initializing a model	4
3.3	Neighbor hopping and the Hamiltonian	5
3.4	Plotting	6
3.5	Wannier90 interface	6
3.6	Band corepresentations	7
4	Examples	7
4.1	Graphene	7
4.1.1	Including spin-orbit coupling	9
4.2	Induced representations in a cubic lattice	10
4.3	A noncollinear magnetic kagome model	12
4.3.1	Viewing the magnetic structure	13
4.3.2	The Hamiltonian and bands	13
5	Frequently asked questions	14

This manual introduces the commonly used features of MagneticTB 2.0. For a first calculation, install the package and follow the graphene example in Section 4.1, running the code in order.

1 Features

MagneticTB is an open-source package based on the corepresentation theory of magnetic groups. It constructs tight-binding models from crystal symmetry and local orbitals. To define a model, provide the lattice vectors, Wyckoff positions, magnetic moments, symmetry operations, and basis functions. With these inputs, the package generates the symmetry-allowed Hamiltonian. You can then assign parameter values and plot the bands. Its features include:

- Obtaining symmetry operations of magnetic space, layer, and rod groups.
- Constructing models of magnetic and nonmagnetic materials, including non-collinear magnetic structures.
- Constructing spinless models and double-group models with spin-orbit coupling.
- Starting from local orbitals at a reference site and using an induced representation to construct basis functions at equivalent sites.
- Plotting crystal structures, magnetic moments, first Brillouin zones, and bands, including interactive band plots with adjustable parameters.
- Reading and writing Wannier90 files, and working with band-fitting and band-corepresentation tools.

2 Installation, updates, and uninstallation

2.1 Installing and loading the package

Once you have a local `.paclet` file, run this command in Mathematica:

```
PacletInstall ["/absolute/path/MagneticTB-2.0.10.paclet"]
```

Replace the path and filename with those of your file. On Windows, forward slashes can also be used in paths, for example `"C:/Users/yourname/Downloads/MagneticTB-2.0.10.paclet"`.

Save your open notebooks, then **quit and reopen Mathematica completely**, rather than only restarting the kernel. After reopening Mathematica, load the package:

```
Needs["MagneticTB"]
```

This loads the functions. You still need to define a model with `init` before calling `symham`.

2.2 Opening the documentation

Search for **MagneticTB** in Mathematica's Documentation Center, or run:

```
SystemOpen["paclet:MagneticTB/guide/MagneticTB"]
```

This opens the help home page shown in Figure 1. Start with *Getting started with MagneticTB* for installation, loading, and a first model. Scroll down to browse the functions by topic.



Figure 1: The MagneticTB help home page in Mathematica, shown with the Simplified Chinese interface. Click a blue link to open a tutorial or function page. (The author purchased the Chinese-language version of Mathematica, so an English-interface screenshot is not available.)

To go directly to the initialization reference page, for example, run:

```
SystemOpen["paclet:MagneticTB/ref/init"]
```

The figures in this manual are static. Use Mathematica to operate the sliders in `bandManipulate`.

2.3 Updating

To update, install a local file with a higher version number:

```
PacletInstall ["/absolute/path/MagneticTB-<new-version>.paclet"]
```

Then save your work, quit Mathematica completely, and reopen it. Use `PacletFind["MagneticTB"]` to see the installed versions. Automatic online updates are not currently provided.

The message `PacletInstall :: samevers` means that this version is already installed. If you need to reinstall the same version, uninstall it first:

```
PacletUninstall ["MagneticTB"]
PacletInstall ["/absolute/path/MagneticTB-2.0.10.paclet"]
```

2.4 Uninstalling

To remove the package, run:

```
PacletUninstall ["MagneticTB"]
```

This uninstalls the package, but does not delete your own model notebooks. Quit and reopen Mathematica afterwards so that you do not continue using definitions already loaded in the kernel.

3 Using MagneticTB

3.1 Obtaining symmetry operations

Three functions form the main modeling workflow: `msgop` obtains the symmetry operations, `init` sets up the model, and `symham` constructs the Hamiltonian.

For a nonmagnetic material, one normally uses a gray group, which includes time-reversal symmetry. For example:

```
1 Needs["MagneticTB"];
2 sgop = msgop[gray[191]];
```

Here `gray[191]` gives the identifier of the gray group associated with space group 191, and `msgop` converts that identifier into the complete list of symmetry operations. It also prints the BNS number, group symbol, and lattice vectors. To specify a magnetic group by its BNS number, such as 191.239, use:

```
msgop[bnsdict[{191, 239}]]
```

Thus, `gray` and `bnsdict` are normally used together with `msgop`. You do not need to memorize the internal identifiers.

Each operation has the form `{label, rotation matrix, translation, "F" or "T"}`. The rotation and translation use fractional coordinates in the lattice basis. "T" denotes an operation containing time reversal, and "F" one without it. If you change the lattice basis, transform the symmetry operations into the same basis; changing only a lattice angle is not sufficient. Magnetic layer and rod groups are used in the same way:

```
mlgop[graylayer[80]]
mrgop[grayrod[75]]
```

These commands obtain the gray groups associated with layer group 80 and rod group 75, respectively.

3.2 Initializing a model

With the symmetry operations available, use `init` to specify the lattice, atomic positions, and basis functions. For graphene, the complete input is:

```
1 sgop = msgop[gray[191]];
2 init [
3   lattice -> {
4     {a, 0, 0}, {-a/2, Sqrt[3] a/2, 0}, {0, 0, c}
5   },
6   lattpar -> {a -> 1, c -> 3},
7   wyckoffposition -> {{{1/3, 2/3, 0}, {0, 0, 0}}},
8   symminformation -> sgop,
9   basisFunctions -> {"pz"}
10 ];
```

The three rows of `lattice` are the real-space lattice vectors. The option `lattpar` assigns numerical values to their parameters. Here the in-plane angle is 120°

and the interlayer spacing is $c = 3$. The option `symminformation` uses the complete operation list obtained above.

The option `wyckoffposition` specifies the inequivalent sites and their magnetic moments, in the form

$$\{\{\mathbf{r}_1, \mathbf{m}_1\}, \{\mathbf{r}_2, \mathbf{m}_2\}, \dots\}.$$

Here $\mathbf{r}_i$ is a fractional position and $\mathbf{m}_i$ is a magnetic moment expressed in the lattice basis. Only one representative of each set of equivalent sites is needed; the package generates the remaining sites using symmetry. The carbon atoms in this example are nonmagnetic, so their moments are written as $\{0,0,0\}$.

The entries of `basisFunctions` correspond to the inequivalent sites in the input. Put the basis functions for each site in a separate list. Common orbitals are written as follows:

Orbital	String	Orbital	String
s	"s"	p_x	"px"
p_y	"py"	p_z	"pz"
$p_x + ip_y$	"px+ipy"	$p_x - ip_y$	"px-ipy"
d_{z^2}	"dz2"	d_{xy}	"dxy"
d_{yz}	"dyz"	d_{xz}	"dxz"
$d_{x^2-y^2}$	"dx2-y2"		

To include spin, append "up" or "dn" to the orbital name. For example, the two spin states of a carbon p_z orbital are:

```
basisFunctions -> {"pzup", "pzdnp"}
```

You can also enter an analytical expression for an orbital that is not listed in the table. For example, an f_{xyz} orbital is written as:

```
basisFunctions -> {{x*y*z}}
```

After initialization, inspect the orbital order of the Hamiltonian and the crystal structure:

```
orbitalTable []
showCrystalStructure []
```

For graphene, the package generates carbon atoms at $(1/3, 2/3, 0)$ and $(2/3, 1/3, 0)$, each with one p_z orbital, giving a 2×2 Hamiltonian. To study a different model, provide a new, complete `init [...]` call. MagneticTB does not reuse the previous model.

3.3 Neighbor hopping and the Hamiltonian

The call `symham[1]` gives the onsite term, `symham[2]` the nearest-neighbor hopping, and `symham[3]` the second-neighbor hopping. Each call computes only the terms at the corresponding distance; it does not automatically add the preceding terms. For example:

```
1 ham = Sum[symham[i], {i, 3}];
2 MatrixForm[ham]
```

This gives the Hamiltonian containing onsite, nearest-neighbor, and second-neighbor terms. The symbols `e1`, `t1`, `r1`, and so on are independent parameters.

Choose their values for the problem of interest, or determine them by fitting first-principles bands.

By default, `init` prepares 10 distance groups, including the onsite term as the first group. To include more distant neighbors, add `InitialBondShells -> 15` to the complete initialization, then call the required `symham[j]`. Ordinary nearest- and second-neighbor examples do not need this option.

By default, `symham` constructs a Hermitian Hamiltonian. For a non-Hermitian model, change the relevant call to `symham[2, "Hermitian" -> False]`. Forward and reverse hoppings are then no longer related by Hermiticity. For a complex spectrum, plot its real part, imaginary part, or complex-plane trajectory as appropriate for the problem.

3.4 Plotting

After running `init`, you can plot the structure and first Brillouin zone, and obtain a conventional path:

```
showCrystalStructure []
showBrillouinZone []
standardKPath[]
```

For magnetic models, the structure plot also shows magnetic-moment arrows. Three-dimensional plots use an orthographic projection by default. Use Mathematica's `Show` to change the viewing direction.

The function `standardKPath` selects a built-in conventional path from the lattice metric. It does not calculate space-group little groups or irreducible representations. For nonstandard bases, centered lattices, or special paths, check the point coordinates and provide your own path if needed.

With a Hamiltonian and path, plot the bands using:

```
bandplot[path, n, ham, parameters]
bandManipulate[path, n, ham]
```

Here `path` specifies the endpoints and labels of each segment, `n` controls the sampling density, and `parameters` assigns parameter values. The function `bandplot` plots bands for fixed parameters, while `bandManipulate` provides sliders. To use an automatically selected path, write:

```
bandManipulate[standardKPath[], 80, ham]
```

Path points use reciprocal fractional coordinates, such as $K = (1/3, 1/3, 0)$. In the default Hamiltonian, the variables `kx`, `ky`, and `kz` include a factor of 2π . To evaluate the Hamiltonian directly at K , substitute `2 Pi {1/3,1/3,0}`. The plotting functions perform this conversion themselves; do not multiply the path by 2π again.

3.5 Wannier90 interface

After evaluating `symham[1]` through `symham[3]`, export these three contributions to a Wannier90 file:

```
hop[3, parameters, "hrExport" -> "/absolute/path/wannier90_hr.dat"]
```

The list `parameters` must assign numerical values to every hopping parameter. This form exports the real-space hoppings already obtained, without an inverse

Fourier transform of $H(\mathbf{k})$. The form `hop[ham, parameters]` is also available for extracting hoppings from an explicit Bloch Hamiltonian. When converting your own Hamiltonian, check the orbital centers and phase convention.

To read the file and recover the Hamiltonian, use:

```
1 hrData = readHR["/absolute/path/wannier90_hr.dat"];
2 buildBlochHamiltonian[hrData, {0, 0, 0}] // MatrixForm
```

The second line displays the Hamiltonian at Γ . Change the three-component fractional coordinate to obtain the matrix at another $\mathbf{k}$ point. The matrix rows and columns follow the orbital order in the file.

3.6 Band corepresentations

Band-corepresentation calculations require a separate installation of `MSGCorep` and its dependency `SpaceGroupRep`. In a fresh kernel, explicitly load the dependency first:

```
Needs["MSGCorep"];
Needs["MagneticTB"];
operations = getMSGElemFromMSGCorep[{164, 87}];
```

Pass `operations` to a complete `init` call in its original order, using the corresponding lattice vectors. After constructing the Hamiltonian, call `getTBBandCorep[BNSNo, ham, parameters, kset]` to print the energies, degeneracies, and corepresentation names at each $\mathbf{k}$ point. Scalar orbitals use single-valued representations; spinor bases such as `{"sup", "sdn"}` use double-valued representations.

This interface applies only to operations obtained from `getMSGElemFromMSGCorep`, not those from `msgop` or other sources. A complete example is given in the *Band corepresentations* tutorial linked from the help home page. The package does not load the dependency for you or replace the external calculation with an internal approximation.

4 Examples

4.1 Graphene

To construct a tight-binding model of graphene near the Fermi level, we need only know that the two carbon atoms occupy the $2c$ Wyckoff position of space group 191, and that the relevant electronic states come from carbon p_z orbitals. The complete input is:

```
1 Needs["MagneticTB"];
2 sgop = msgop[gray[191]];
3 init [
4   lattice -> {
5     {a, 0, 0}, {-a/2, Sqrt[3] a/2, 0}, {0, 0, c}
6   },
7   lattpar -> {a -> 1, c -> 3},
8   wyckoffposition -> {{{1/3, 2/3, 0}, {0, 0, 0}}},
9   symminformation -> sgop,
10  basisFunctions -> {"pz"}
11 ];
```

Line 2 obtains all operations of the gray group associated with space group 191. Only one carbon atom is supplied to `wyckoffposition`; the package generates its equivalent partner. To see the in-plane arrangement clearly, plot a top view of 3×3 primitive cells:

```
1 Show[
2   showCrystalStructure[
3     "CellRange" -> {{0, 2}, {0, 2}, {0, 0}}, ImageSize -> 440
4   ],
5   ViewPoint -> {0, 0, 3}, ViewVertical -> {0, 1, 0}
6 ]
```

The option "CellRange" changes only the cells shown in the picture, not the size of the Hamiltonian. The spheres are carbon atoms. The gray lines are cell boundaries, not hopping bonds.

For the in-plane bands, use the Γ - M - K - Γ path. These are the first three segments of the hexagonal path:

```
1 path = Take[standardKPath[], 3];
2 showBrillouinZone["KPath" -> path, ImageSize -> 440]
```

The endpoints are $\Gamma = (0, 0, 0)$, $M = (1/2, 0, 0)$, $K = (1/3, 1/3, 0)$, and Γ , in that order. The colored lines in the Brillouin-zone plot show the same path used for the band plot below.

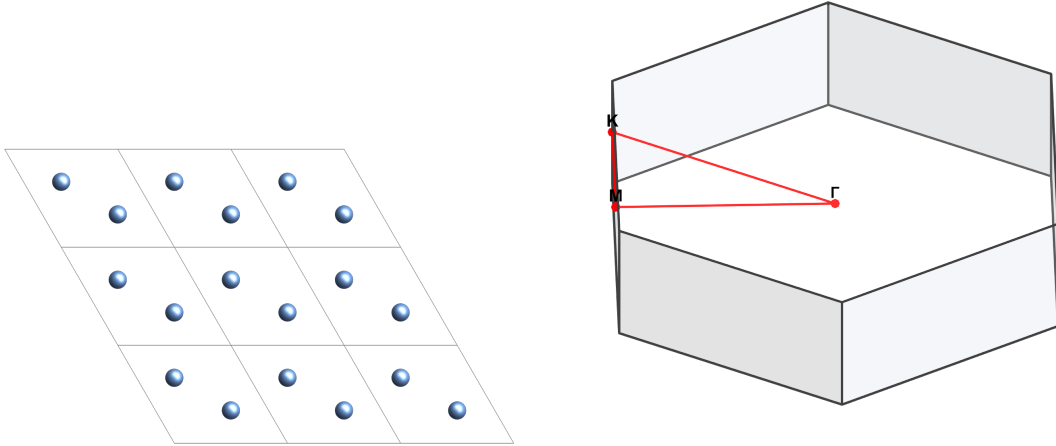


Figure 2: Atomic arrangement of graphene (left) and its first Brillouin zone (right). The top view contains 18 carbon atoms. The path in the right panel lies in the $k_z = 0$ plane.

After checking the structure, generate the Hamiltonian with onsite, nearest-neighbor, and second-neighbor terms:

```
1 ham = Sum[symham[i], {i, 3}];
2 MatrixForm[ham]
```

The two rows correspond to the p_z orbitals of the two carbon atoms. The diagonal entries contain the onsite energy and second-neighbor hoppings within each sublattice. The off-diagonal entries come from nearest-neighbor hoppings between the two sublattices. Here `e1` is the common onsite energy, while `t1` and `r1` control the nearest- and second-neighbor hoppings, respectively.

Combining the exponential terms gives

$$H(\mathbf{k}) = \begin{pmatrix} d(\mathbf{k}) & t_1 f(\mathbf{k}) \\ t_1 f(\mathbf{k})^* & d(\mathbf{k}) \end{pmatrix},$$

where

$$\begin{aligned} d(\mathbf{k}) &= e_1 + 2r_1[\cos k_x + \cos k_y + \cos(k_x + k_y)], \\ f(\mathbf{k}) &= e^{i(k_x - k_y)/3}(1 + e^{-ik_x} + e^{ik_y}). \end{aligned}$$

The three exponential terms come from the three nearest-neighbor bonds. They cancel at K , where the two bands cross.

Once the parameters have been assigned, a single plotting command gives the band structure:

```
1 parameters = {e1 -> 0.05, t1 -> 0.5, r1 -> 0.02};
2 bandplot[path, 80, ham, parameters, ImageSize -> 440, FontSize -> 16]
```

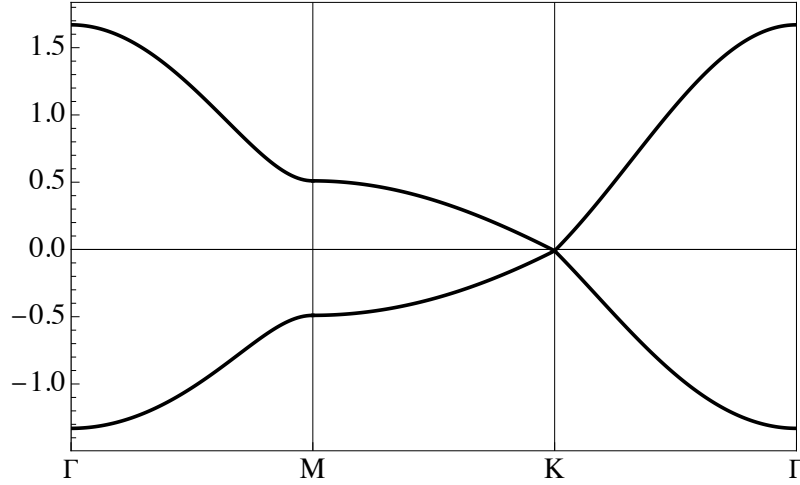


Figure 3: The two p_z bands of graphene for $e_1 = 0.05$, $t_1 = 0.5$, and $r_1 = 0.02$. The bands cross at K . Second-neighbor hopping changes the dispersion but does not open a gap at this crossing.

The energy unit follows the parameter units: parameters entered in eV give energies in eV. To explore how the parameters affect the bands, run:

```
bandManipulate[path, 80, ham]
```

Adjust the t_1 and r_1 sliders in the notebook to see the changes.

The model can also be exported directly:

```
hop[3, parameters, "hrExport" -> "/absolute/path/graphene_hr.dat"]
```

There is no need to print the entire hopping table. If you want to inspect the atoms and distances for a particular neighbor group, use `showbonds[2]` or `showbonds[3]` separately.

4.1.1 Including spin-orbit coupling

To include spin-orbit coupling (SOC), replace each carbon p_z orbital with the two basis functions $p_z \uparrow$ and $p_z \downarrow$. The complete initialization is:

```

1 init [
2   lattice -> {
3     {a, 0, 0}, {-a/2, Sqrt[3] a/2, 0}, {0, 0, c}
4   },
5   lattpar -> {a -> 1, c -> 3},
6   wyckoffposition -> {{{1/3, 2/3, 0}, {0, 0, 0}}},
7   symminformation -> sgop,
8   basisFunctions -> {"pzup", "pzdn"}
9 ];
10 hamsoc = Sum[symham[i], {i, 3}];
11 MatrixForm[hamsoc]

```

The matrix now has four rows: the two spin states of the first carbon atom, followed by those of the second. The lattice, positions, and space group are unchanged, but the package constructs double-group representations in the spinor basis. Simply copying the earlier two-band Hamiltonian twice does not produce the SOC model.

In this four-band model, $r1$ controls spin-dependent second-neighbor hopping and $r2$ controls spin-independent second-neighbor hopping. Note that $r1$ no longer has the meaning it had in the two-band model. Compare the bands with the SOC parameter set to 0 and 0.05:

```

1 bandplot[
2   path, 80, hamsoc, {e1 -> 0.05, t1 -> 0.5, r1 -> 0, r2 -> 0.02},
3   ImageSize -> 440, FontSize -> 16
4 ]
5 bandplot[
6   path, 80, hamsoc, {e1 -> 0.05, t1 -> 0.5, r1 -> 0.05, r2 -> 0.02},
7   ImageSize -> 440, FontSize -> 16
8 ]

```

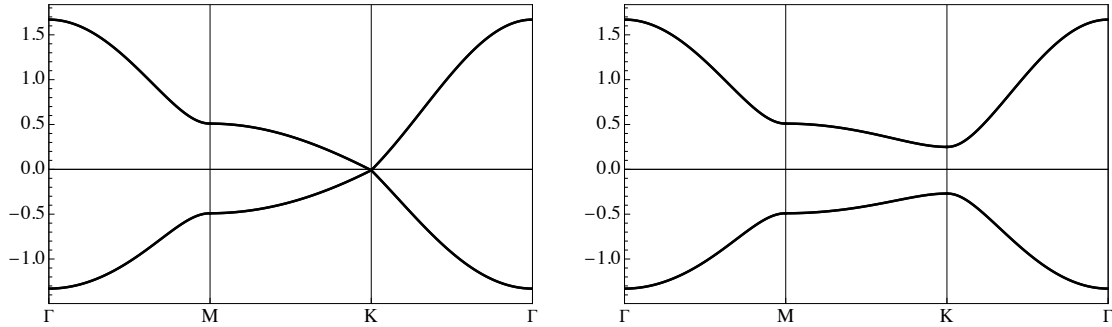


Figure 4: The same four-band graphene model with the SOC parameter set to 0 (left) and 0.05 (right). SOC opens a gap at K in the right panel. Each visible band remains doubly degenerate.

4.2 Induced representations in a cubic lattice

In some models, we want to retain only one p orbital along a local axis at each atom. Cubic rotations, however, can transform p_x into p_y or p_z . If the same fixed local basis is used at every atom, all three orbitals p_x, p_y, p_z must be supplied. The induced mode instead starts from a single p_x at a reference atom and uses symmetry to determine the orbitals at the equivalent atoms.

For example, the position $(1/2, 0, 0)$ in space group 221 generates three equivalent sites. To retain only p_x at the reference site, use:

```
1 Needs["MagneticTB"];
2 init [
3   lattice -> {{a, 0, 0}, {0, a, 0}, {0, 0, a}},
4   lattpar -> {a -> 1},
5   wyckoffposition -> {{{1/2, 0, 0}, {0, 0, 0}}},
6   symminformation -> msgop[gray[221]],
7   basisFunctions -> {"px"},
8   RepresentationMode -> "Induced"
9 ];
10 orbitalTable []
```

The option `RepresentationMode -> "Induced"` selects the induced mode. The three rows of the orbital table are, in order, p_x at $(1/2, 0, 0)$, p_z at $(0, 0, 1/2)$, and p_y at $(0, 1/2, 0)$. The same p_x orbital has not been placed on all three atoms. Cubic rotations determine the orbital directions at the other two sites.

Plot the structure and the bands of the nearest-neighbor model:

```
1 showCrystalStructure[
2   "CellRange" -> {{0, 1}, {0, 1}, {0, 1}}, ImageSize -> 440
3 ]
4 hamCubic = Simplify[ComplexExpand[symham[1] + symham[2]]];
5 hamCubic // MatrixForm
6 pathCubic = standardKPath[];
7 bandplot[
8   pathCubic, 60, hamCubic, {e1 -> 0, t1 -> -1},
9   ImageSize -> 440, FontSize -> 16
10 ]
```

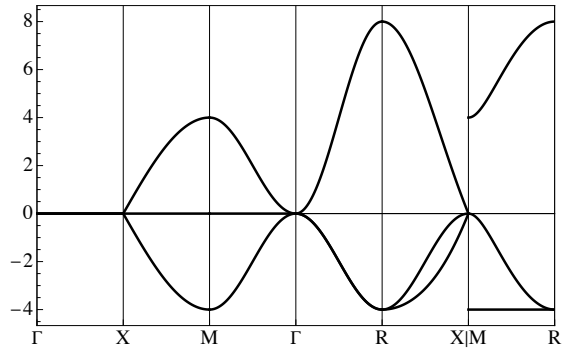
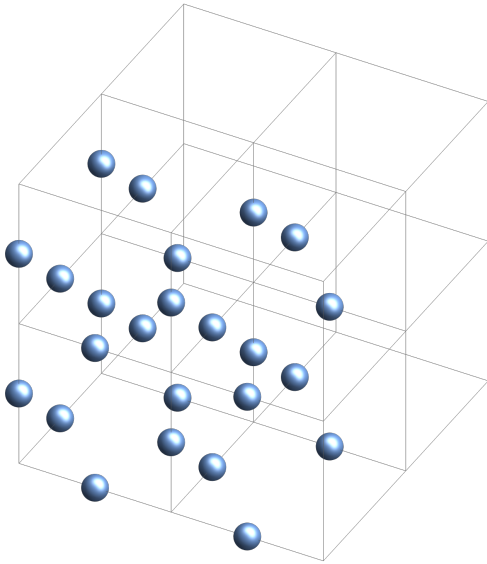


Figure 5: Equivalent atoms in the cubic induced model (left) and its three bands (right). Each atom carries one orbital obtained by transporting the reference p_x . The structure plot shows atomic positions, not orbital lobes.

In the orbital order listed above, the matrix is

$$H(\mathbf{k}) = e_1 I_3 - 4t_1 \begin{pmatrix} 0 & s_x s_z & s_x s_y \\ s_x s_z & 0 & s_y s_z \\ s_x s_y & s_y s_z & 0 \end{pmatrix}, \quad s_i = \sin(k_i/2).$$

All three bands therefore coincide at e_1 along Γ - X . Along X - M , two bands disperse upwards and downwards, while the third remains at e_1 . The label $X|M$ marks a jump from X to M before the next segment; those two points are not connected.

To use the default "DirectProduct" mode instead, replace the last two options in the same complete initialization with:

```
basisFunctions -> {"px", "py", "pz"},
RepresentationMode -> "DirectProduct"
```

Now each of the three atoms has three orbitals, giving a nine-band model. This is not the same basis as the three-band model above, so the complete band structures should not be expected to match. Induction lets you supply only the local states needed at the reference site; it does not arbitrarily remove six bands from a nine-band model.

If another program already supplies the local representation matrices, use `initfromrep`. This function accepts representation matrices rather than orbital strings. It requires a complete group in a fixed order and does not accept `GenerateSymmetryGroup`. It is not needed for ordinary orbital-based modeling.

4.3 A noncollinear magnetic kagome model

Consider a kagome model with three in-plane magnetic moments separated by 120° . Their sum is zero, but each atom has a nonzero moment, so this is not a nonmagnetic model. Use magnetic space group 191.239 and supply a local spinor at the reference site:

```
1 Needs["MagneticTB"];
2 init [
3   lattice -> {
4     {a, 0, 0}, {-a/2, Sqrt[3] a/2, 0}, {0, 0, c}
5   },
6   lattpar -> {a -> 1, c -> 4},
7   wyckoffposition -> {{{1/2, 0, 0}, {1, 0, 0}}},
8   symminformation -> msgop[typelll[{191, 239}]],
9   basisFunctions -> {{{1, -1}/Sqrt[2]}},
10  RepresentationMode -> "Induced"
11 ];
```

Here `typelll[{191,239}]` specifies this type-III magnetic space group by its BNS number. The two components of $\{1, -1\}/\text{Sqrt}[2]$ are expressed in the spin $\uparrow, \downarrow$ basis. Together they describe **one** local state, not two independent basis functions. Induction transports this state to the other two equivalent sites, giving three bands rather than the full six-band spinful model.

The generated positions are $(1/2, 0, 0)$, $(0, 1/2, 0)$, and $(1/2, 1/2, 0)$, in that order. Their moments, expressed in the lattice basis, are $(1, 0, 0)$, $(0, 1, 0)$, and $(-1, -1, 0)$. Since the in-plane lattice vectors form an angle of 120° , these moments have equal Cartesian lengths and are separated by 120° .

4.3.1 Viewing the magnetic structure

First plot 2×2 primitive cells, then a top view of 3×3 cells:

```

1 showCrystalStructure[
2   "CellRange" -> {{0, 1}, {0, 1}, {0, 0}},
3   "MomentScale" -> 0.3, ImageSize -> 400
4 ]
5 Show[
6   showCrystalStructure[
7     "CellRange" -> {{0, 2}, {0, 2}, {0, 0}},
8     "MomentScale" -> 0.3, ImageSize -> 400
9   ],
10  ViewPoint -> {0, 0, 3}, ViewVertical -> {0, 1, 0}
11 ]

```

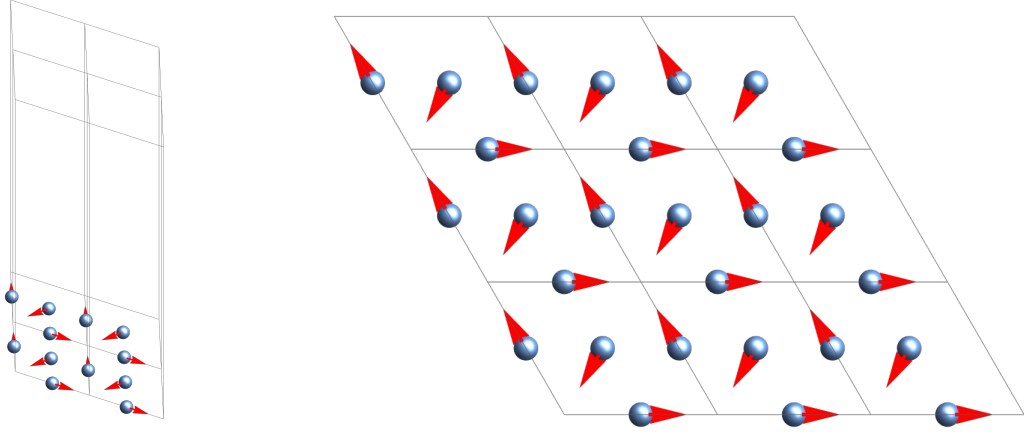


Figure 6: Oblique view (left) and top view (right) of the noncollinear kagome magnetic structure. The red arrows come directly from the initialized magnetic moments. The three moment directions repeat throughout the top view.

The option "MomentScale" changes only the displayed arrow length, not the magnetic moments in the model. Again, gray lines are cell boundaries, not hopping paths.

4.3.2 The Hamiltonian and bands

This example includes only onsite and nearest-neighbor terms:

```

1 kagomeHamiltonian = Simplify[ComplexExpand[symham[1] + symham[2]]];
2 kagomeHamiltonian // MatrixForm

```

For real momenta, ComplexExpand and Simplify rewrite pairs of exponentials as cosines without changing the model. The matrix is

$$H(\mathbf{k}) = \begin{pmatrix} e_1 & 2t_1 \cos \frac{k_x + k_y}{2} & 2t_1 \cos \frac{k_y}{2} \\ 2t_1 \cos \frac{k_x + k_y}{2} & e_1 & -2t_1 \cos \frac{k_x}{2} \\ 2t_1 \cos \frac{k_y}{2} & -2t_1 \cos \frac{k_x}{2} & e_1 \end{pmatrix}.$$

The equal diagonal entries give the common onsite energy of the three equivalent sites. The relative minus sign in the off-diagonal entries follows from the chosen

induced spinor basis; it must not be removed merely to make the matrix look more uniform.

Use the Γ - M - K - Γ path in the $k_z = 0$ plane to plot the Brillouin zone and bands:

```
1 planePath = Take[standardKPath[], 3];
2 showBrillouinZone["KPath" -> planePath, ImageSize -> 400]
3 bandplot[
4   planePath, 60, kagomeHamiltonian, {e1 -> 0, t1 -> -1},
5   ImageSize -> 400, FontSize -> 16
6 ]
```

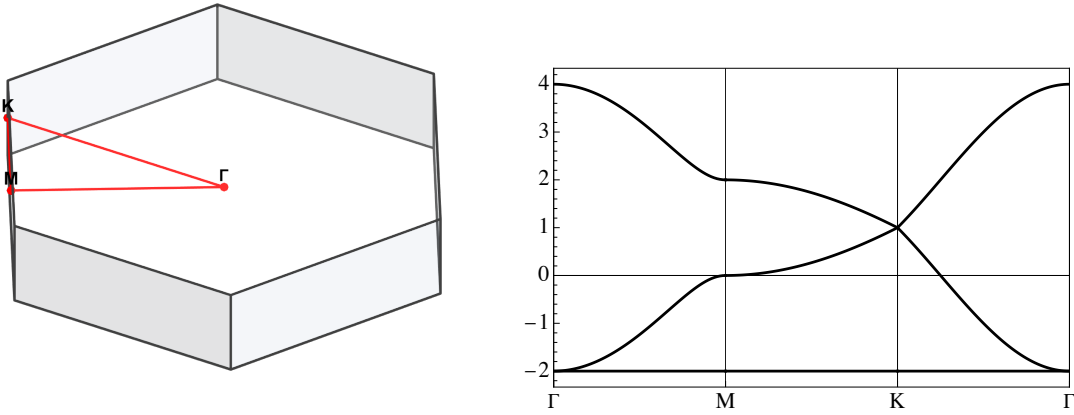


Figure 7: In-plane path (left) and bands (right) of the same kagome model. The flat band lies at $E = -2$, and the two dispersive bands meet at K at $E = 1$.

At Γ , the flat band touches the lower dispersive band. These two contributions contain no interlayer hopping, so the Hamiltonian does not depend on k_z . This is an ideal magnetic kagome model. Its parameters illustrate the symmetry-allowed bands; they are not a fit to a particular material.

5 Frequently asked questions

1. What if the help pages are not available after installation?

Answer: Use `PacletFind["MagneticTB"]` to confirm the installation, save your work, then quit and reopen Mathematica completely. Run `SystemOpen["paclet:MagneticTB/guide/MagneticTB"]` to open the help home page. Restarting only the kernel can leave old help pages open in the front end.

2. Why can `init` not construct a model from lattice constants alone?

Answer: Lattice constants set the geometric scale, but do not specify the atomic positions, symmetry, or orbitals. Provide all five model inputs as in Section 3.2. A call such as `init[lattice -> ...]` is not a complete model.

3. Do operations obtained from `msgop` need group completion?

Answer: No. The function `msgop` already provides the complete operation

list, and `GenerateSymmetryGroup` defaults to `False`. Add `GenerateSymmetryGroup` $\rightarrow$ `True` to the complete initialization only when supplying generators yourself.

4. Why are all bands horizontal when `bandManipulate` first opens?
Answer: The sliders start at zero. With zero onsite energies and hoppings, the Hamiltonian is the zero matrix. Set a hopping parameter to a nonzero value, or use `bandplot` with explicit parameter values, to see the dispersion.
5. Why does the Hamiltonian expression change with the lattice basis?
Answer: Momentum coordinates, fractional atomic positions, and symmetry operations all depend on the basis. For example, using in-plane angles of 120° and 60° for a hexagonal lattice produces different expressions for the same physical model. Before comparing results, put the lattice, positions, operations, and momentum path into the same convention. Comparing only parameter names or individual matrix elements is not sufficient.
6. When can the induced mode start from a single p_x orbital?
Answer: The local basis at the reference site must be closed under its site-symmetry group. The cubic example in Section 4.2 satisfies this condition and can start from one p_x orbital. A single component is not sufficient at every site for every orbital. If a site-symmetry operation maps p_x to another independent state, include that state in the reference basis as well.
7. Can I remove a few parameters to keep spin but neglect SOC?
Answer: Parameter names alone do not determine which terms to remove. `MagneticTB` supports separate spatial and spin actions, and the explicit-representation interface can include one internal continuous spin rotation C_∞ . A model without SOC should use spin-symmetry constraints appropriate to the magnetic structure. See the *Continuous spin symmetry* tutorial for the construction, rather than setting arbitrary entries of an SOC Hamiltonian to zero.
8. How can I calculate other physical properties?
Answer: Use the relevant property functions in the package, or export a Wannier90 file with `hop` for use in other software. Functions can differ in their coordinate and orbital-center conventions. Read the relevant help page first, and do not mix a hand-written momentum function with the default $H(\mathbf{k})$ convention.
9. How do I use version 1.0?
Answer: In a fresh kernel, run `Needs["MagneticTBOld"]`, then use `initold`, `symhamold`, and the other version-1.0 functions. Load versions 1.0 and 2.0 in separate kernels. See the *Using MagneticTB 1.0* tutorial for complete inputs.

10. Where can I find more examples?

Answer: After installing the packet, open the *MagneticTB general examples*, *Crystal structure*, *Brillouin zone*, and *k paths*, or band-fitting tutorials from the help home page. Function pages also show how to use their options. You can adapt the example parameters to your own model.

11. How can I report a problem?

Answer: Join QQ group 625192239, open an issue at <https://github.com/zhangzeyingv/MagneticTB/issues>, or email zhangzeyingv@gmail.com. Include the package version, complete input, and error messages so that the problem can be reproduced.



12. How should I cite MagneticTB?

Answer: If you use the package in your research, please cite: Zeying Zhang, Zhi-Ming Yu, Gui-Bin Liu, Yugui Yao, *MagneticTB: A package for tight-binding model of magnetic and non-magnetic materials*, Computer Physics Communications **270**, 108153 (2022). DOI: 10.1016/j.cpc.2021.108153.

```
@article {ZHANG2022108153,
  title = {MagneticTB: A package for tight-binding model
    of magnetic and non-magnetic materials},
  author = {Zhang, Zeying and Yu, Zhi-Ming and
    Liu, Gui-Bin and Yao, Yugui},
  journal = {Computer Physics Communications},
  volume = {270},
  pages = {108153},
  year = {2022},
  doi = {10.1016/j.cpc.2021.108153}
}
```
